KrishiBandhu

AI-Powered Farm-to-Market Intelligence Platform

Team: Tech Titans
College: R C Patel Institute of Technology, Shirpur
Date: March 26, 2026
Theme: Agriculture Technology / Smart Farming

Table of Contents

- 1. Executive Summary
- 2. Project Overview
- 3. Problem Statement & Solution
- 4. System Architecture
- 5. Technology Stack
- 6. Development Roadmap
- 7. Installation & Setup Guide
- 8. API Documentation
- 9. Database Schema
- 10. Feature Documentation
- 11. Frontend Components
- 12. Backend Implementation
- 13. Machine Learning Model
- 14. Deployment Guide
- 15. Testing Guide
- 16. Troubleshooting
- 17. Future Enhancements
- 18. Team Information
- 19. References

1. Executive Summary

KrishiBandhu is an AI-powered farm-to-market digital intelligence platform designed to address the critical information gap faced by Indian farmers at harvest time. By integrating real-time mandi prices, weather data, and demand signals with predictive analytics, KrishiBandhu empowers farmers to make informed decisions about when and where to sell their produce, maximizing profits while ensuring fair and transparent transactions.

The platform bridges the gap between farmers and buyers by providing:

- Real-time price forecasts using machine learning
- Optimal selling time recommendations (Sell Now vs Wait)
- Demand heatmap intelligence across markets
- Buyer trust scores for transparent transactions
- Multi-language support (English, Hindi, Marathi)
- Voice-first accessibility for low-literacy farmers

2. Project Overview

2.1 Vision

To transform agricultural trade by empowering farmers with market intelligence, ensuring fair prices, reducing post-harvest losses, and building trust in rural digital ecosystems.

2.2 Mission

Develop an accessible, intelligent platform that provides farmers with real-time price insights, demand forecasts, and smart recommendations while building a transparent marketplace connecting farmers directly with buyers.

2.3 Project Scope

- Price prediction for major crops (wheat, jowar, sugarcane, vegetables)
- Coverage of major mandi markets in India (initial focus: Nashik, Maharashtra)
- Farmer mobile/web app with dashboard
- Buyer marketplace app
- Admin panel for system management
- Machine learning backend for predictions
- Integration with government agricultural data

2.4 Key Objectives

- Increase farmer income by 15-25% through better pricing decisions
- Reduce post-harvest losses by 10-15% through optimal timing
- Build farmer confidence in digital platforms through trust mechanisms
- Create a replicable model for other states/regions
- Achieve 80%+ accuracy in price predictions
- Support 10,000+ farmers in pilot phase

3. Problem Statement & Solution

3.1 The Problem: "Who Grows the Farmer's Income?"

Current Situation:

- Farmers lack real-time market price information
- Middlemen exploit information asymmetry, paying 30-40% less than market rates
- Price volatility creates decision paralysis at harvest time
- Farmers don't know buyer credibility or trustworthiness
- Post-harvest losses occur due to poor timing and storage decisions
- Digital platforms remain inaccessible due to language and literacy barriers

Impact: A farmer growing wheat with a 15,000 rupee investment per acre may receive only 1800-2000 Rs/quintal from middlemen, while buyers in the market pay 2400-2600 Rs/quintal. This 20-30% loss compounds over a season, keeping farmers in a cycle of low income.

3.2 The Solution: KrishiBandhu Platform

A multi-layered solution combining data, AI, and transparency:

Data Layer: Real-time mandi prices, weather forecasts, demand signals, historical trends

AI Layer: Machine learning models for price prediction, demand forecasting, risk assessment

Intelligence Layer: Smart "Sell Now vs Wait" recommendations, profit optimization, trust scoring

Access Layer: Mobile app, voice support, multi-language UI designed for low-literacy users

Trust Layer: Buyer verification, transaction records, rating system, secure payments

3.3 Expected Outcomes

- Farmers increase income by 15-25% through better pricing
- Reduce post-harvest losses by 10-15%
- 50%+ farmers adoption in pilot region
- Elimination of middlemen for direct farmer-buyer transactions
- Model scalability to other states and crops

4. System Architecture

4.1 Architecture Overview

KrishiBandhu follows a modular, API-driven microservices architecture designed for scalability, maintainability, and independent deployment of components.

4.2 Core Components

Frontend Layer

- Farmer Mobile App (React Native)
- Farmer Web App (React.js)
- Buyer App
- Admin Dashboard

API Gateway

- REST API
- Rate limiting
- Authentication
- Load balancing

Business Logic Layer

- Price Prediction Engine
- Recommendation System
- Trust Scoring
- Demand Forecasting

Data Processing Layer

- Data Cleaning
- Feature Engineering
- ML Model Training
- Caching Layer (Redis)

Storage Layer

- Primary Database (PostgreSQL)
- Cache (Redis)
- File Storage (S3)

External Integrations

- Mandi APIs
- Weather APIs
- Payment Gateway

- SMS/Push Notifications

4.3 Data Flow

Step-by-step data processing flow:

1. Data Collection: Real-time mandi prices, weather, demand signals collected via APIs

2. Data Cleaning: Outlier detection, missing value imputation, normalization

3. Feature Engineering: Price trends, seasonality, volatility, demand indicators

4. Model Inference: ML models predict 7-day price forecast

5. Logic Application: Profit optimization, recommendation generation

6. Caching: Results cached for quick retrieval

7. API Response: Insights delivered to frontend apps

8. User Feedback: Farmer actions feed back into model for continuous learning

5. Technology Stack

Component	Technology	Rationale
Frontend	React.js / Next.js	SEO, SSR, component reusability, large ecosystem
Mobile	React Native / Flutter	Cross-platform iOS & Android from single codebase
Styling	Tailwind CSS	Utility-first, responsive, quick prototyping
Charts	Recharts / Chart.js	Interactive visualizations for price trends
Language Support	i18n-next	Multi-language UI (EN, HI, MR)
Backend	Node.js + Express	JavaScript ecosystem, high throughput, async
ML/Data Processing	Python + FastAPI	Industry standard for ML, sci-kit learn, TensorFlow
Database	PostgreSQL	ACID compliance, complex queries, GIS support for location data
Cache	Redis	Fast in-memory caching for price data, real-time updates
ML Libraries	scikit-learn, TensorFlow, Pandas	Proven ML algorithms, data manipulation
Authentication	JWT + bcrypt	Stateless, secure, industry standard
Payments	Razorpay / PhonePe	India-specific payment gateway
Cloud Hosting	AWS / GCP / Heroku	Scalability, reliability, managed services
Containerization	Docker + Kubernetes	Reproducible environments, orchestration
CI/CD	GitHub Actions / GitLab CI	Automated testing, deployment pipelines
API Documentation	Swagger / OpenAPI	Auto-generated, interactive docs
Monitoring	Prometheus + Grafana	Real-time metrics, alerting

6. Development Roadmap

6.1 Phase 1: MVP (Weeks 1-2)

Deliverables:

- Basic farmer dashboard with mock price data
- Simple 7-day price forecast (static)
- Sell Now / Wait recommendation button
- Basic profit calculator
- Static buyer app interface

Tech Focus:

- Frontend setup (React)
- Basic backend API (Express)
- Database schema design
- Mock data generators

Success Metrics:

- Dashboard displays correctly on web/mobile
- All API endpoints functional
- Recommendation logic works

6.2 Phase 2: Core Features (Weeks 3-4)

Deliverables:

- Integration with real mandi price APIs (Agmarknet)
- Weather data integration (OpenWeather API)
- Demand heatmap visualization
- Buyer trust score calculation
- Transaction history logging

Tech Focus:

- API integrations
- Data visualization (Recharts)
- Database optimization
- Caching strategy (Redis)

Success Metrics:

- 99%+ API uptime
- <500ms response times
- Real data flowing end-to-end

6.3 Phase 3: AI & Intelligence (Week 5)

Deliverables:

- ML price prediction model (Linear Regression / ARIMA)
- Historical accuracy testing (80%+ target)
- Profit optimization engine
- Demand forecasting module
- A/B testing framework

Tech Focus:

- ML model training
- Feature engineering
- Model evaluation metrics
- Production ML pipeline

Success Metrics:

- 80%+ prediction accuracy on test set
- Model training time <5 minutes
- Real-time inference <100ms

6.4 Phase 4: Polish & Deployment (Week 6)

Deliverables:

- Multi-language UI (Hindi, Marathi)
- Mobile responsiveness optimization
- Voice input support (Web Speech API)
- Cloud deployment (Vercel/Heroku)
- Admin dashboard for monitoring

Tech Focus:

- i18n implementation
- Performance optimization
- DevOps & deployment
- Documentation

Success Metrics:

- Lighthouse score >90
- Mobile FCP <2s
- Zero critical bugs in testing

7. Installation & Setup Guide

7.1 Prerequisites

- Node.js v16+ and npm
- Python 3.8+ and pip
- PostgreSQL 12+
- Redis 6+
- Git
- Docker (optional, for containerization)

7.2 Frontend Setup (React)

Step 1: Create new Next.js project

```
npx create-next-app@latest krishibandhu
cd krishibandhu
```

Step 2: Install dependencies

```
npm install axios recharts
npm install next-i18next
npm install tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

Step 3: Create project structure

```
mkdir -p pages/api components/farmer components/buyer
mkdir -p styles utils public/locales
mkdir -p pages/admin
```

7.3 Backend Setup (Node.js + Express)

Step 1: Create Express server

```
mkdir krishibandhu-backend
cd krishibandhu-backend
npm init -y
npm install express cors dotenv pg redis axios
```

Step 2: Create environment file (.env)

```
DATABASE_URL=postgresql://user:password@localhost:5432/krishibandh
u
REDIS_URL=redis://localhost:6379
OPENWEATHER_API_KEY=your_api_key
AGMARKNET_API_KEY=your_api_key
JWT_SECRET=your_secret_key
NODE_ENV=development
PORT=5000
```

7.4 Database Setup (PostgreSQL)

Step 1: Create database

```
createdb krishibandhu
```

Step 2: Run migration scripts (see section 9)

```
psql krishibandhu < migrations/001_initial_schema.sql
```

7.5 ML Backend Setup (Python)

Step 1: Create Python environment

```
python3 -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
pip install -r requirements.txt
```

Step 2: Train models

```
python train_price_model.py
python train_demand_model.py
```

8. API Documentation

8.1 Authentication Endpoints

Method	Endpoint	Description	Request Body/Params
POST	/api/auth/register	Register new farmer/buyer	{ email, password, role }
POST	/api/auth/login	User login	{ email, password }
POST	/api/auth/logout	User logout	{ token }
GET	/api/auth/refresh	Refresh JWT token	Authorization header

8.2 Price Data Endpoints

Method	Endpoint	Description	Params
GET	/api/prices/:crop	Get current & historical prices	Query: days=30
GET	/api/forecast/:crop	Get 7-day price forecast	Query: region=Nashik
GET	/api/mandi/:crop	Get prices by specific mandi	Query: mandi_id
POST	/api/user-input	Log farmer's price input	{ crop, quantity, date }

8.3 Recommendation Endpoints

Method	Endpoint	Description	Params
GET	/api/recommendation/:crop	Get Sell Now vs Wait advice	Query: farmer_id
GET	/api/profit-estimate	Calculate potential profit	Query: crop, quantity, current_price
POST	/api/feedback	Log user feedback on recommendation	{ recommendation_id, outcome }

8.4 Buyer Endpoints

Method	Endpoint	Description	Params
GET	/api/buyers	List all verified buyers	Query: region, rating
GET	/api/buyer/:id	Get buyer details & trust score	buyer_id
POST	/api/buyer/verify	Request buyer verification	{ buyer_id, documents }
POST	/api/buyer/rate	Rate a buyer	{ buyer_id, rating, feedback }

8.5 Sample API Responses

Price Forecast Response:

```
{
  "crop": "wheat",
  "current_price": 2150,
  "forecast": [2180, 2200, 2220, 2250, 2280, 2300, 2320],
  "days": ["2024-03-27", "2024-03-28", "2024-03-29", ...],
  "confidence": 0.82,
  "volatility": "medium",
  "last_updated": "2024-03-26T10:30:00Z"
}
```

Recommendation Response:

```
{
  "recommendation": "WAIT",
  "reason": "Price forecast shows 12% increase over 5 days",
  "confidence": 0.85,
  "expected_price": 2410,
  "optimal_sell_date": "2024-03-31",
  "profit_gain": 4260,
  "risk_level": "low"
}
```

9. Database Schema

9.1 Core Tables

Users Table:

```
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(20) NOT NULL,  -- 'farmer', 'buyer', 'admin'
  name VARCHAR(255),
  phone VARCHAR(15),
  region VARCHAR(100),
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);
```

Farmers Table:

```
CREATE TABLE farmers (
  id SERIAL PRIMARY KEY,
  user_id INT REFERENCES users(id),
  village VARCHAR(255),
  crops TEXT[],  -- Array of crop types
  land_area DECIMAL(10, 2),  -- in acres
  total_transactions INT DEFAULT 0,
  rating DECIMAL(3, 2) DEFAULT 0,
  trust_score INT DEFAULT 50,
  last_active TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW()
);
```

Prices Table:

```
CREATE TABLE prices (
  id SERIAL PRIMARY KEY,
  crop VARCHAR(50),
  mandi VARCHAR(100),
  price DECIMAL(10, 2),
  date DATE,
  volume INT,
  source VARCHAR(50),  -- 'agmarknet', 'farmer_input'
  created_at TIMESTAMP DEFAULT NOW(),
  UNIQUE(crop, mandi, date)
);
```

Recommendations Table:

```
CREATE TABLE recommendations (
  id SERIAL PRIMARY KEY,
  farmer_id INT REFERENCES farmers(id),
  crop VARCHAR(50),
  recommendation VARCHAR(20),  -- 'SELL_NOW', 'WAIT'
  expected_price DECIMAL(10, 2),
```

```
  confidence DECIMAL(3, 2),
  created_at TIMESTAMP DEFAULT NOW(),
  outcome VARCHAR(20),  -- 'accepted', 'rejected', 'followed'
  actual_price DECIMAL(10, 2),
  feedback_date TIMESTAMP
);
```

Buyers Table:

```
CREATE TABLE buyers (
  id SERIAL PRIMARY KEY,
  user_id INT REFERENCES users(id),
  company_name VARCHAR(255),
  verified BOOLEAN DEFAULT FALSE,
  trust_score INT DEFAULT 50,
  total_transactions INT DEFAULT 0,
  average_rating DECIMAL(3, 2) DEFAULT 0,
  payment_days INT DEFAULT 7,
  created_at TIMESTAMP DEFAULT NOW()
);
```

10. Feature Documentation

10.1 Price Prediction Feature

How It Works:

- Data Collection: Historical prices (30 days) for each crop-mandi combination
- Feature Engineering: Price trends, seasonality, volatility, moving averages
- Model Training: Linear Regression or ARIMA models trained on historical data
- Inference: 7-day predictions generated daily at 6 AM
- Caching: Results cached in Redis for quick access

Expected Accuracy: 75-85% (Mean Absolute Percentage Error)

10.2 Recommendation Engine

Sell Now vs Wait Logic:

```
def get_recommendation(crop, current_price, forecast):
    price_change = (forecast[0] - current_price) / current_price *
100
    volatility = calculate_volatility(forecast)

    if price_change > 10 and volatility < 5:
        return "WAIT for better price", 85
    elif volatility > 15:
        return "SELL NOW - reduce risk", 75
    else:
        return "SELL NOW - fair price", 80
```

10.3 Trust Score Calculation

Factors (weighted):

Transaction Count (30%): More transactions = higher trust

On-time Payment Rate (30%): 100% on-time = max score

Average Rating (25%): Based on farmer/buyer feedback (1-5 stars)

Account Age (15%): Longer history = higher trust

10.4 Demand Heatmap

Visual representation of crop demand across days and regions. Green = high demand, Yellow = medium, Red = low demand. Helps farmers identify peak demand periods for higher prices.

11. Frontend Components

11.1 Farmer App Components

Dashboard: Home screen with price summary, alerts, recommendations

Price Chart: Interactive 30-day historical + 7-day forecast visualization

Recommendation Card: Large CTA showing "Sell Now" or "Wait for Better Price"

Profit Calculator: Input quantity → shows potential earnings

Buyer Marketplace: List of verified buyers with trust scores

Transaction History: Past sales records with prices, dates, buyers

Alerts: Price threshold notifications, demand alerts

Profile: Farmer details, crops, verification status

11.2 Buyer App Components

Farmer Listings: Search & filter available suppliers by crop, quantity, region

Price Comparison: Compare prices across different farmers

Trust Dashboard: View trust scores, ratings, transaction history

Order Placement: Create & manage purchase orders

Payment Gateway: Secure payment processing (Razorpay)

Logistics: Calculate delivery costs and estimated arrival

Feedback: Rate farmers after transaction

11.3 Multi-Language Support

Supported languages: English, Hindi, Marathi

Implementation using i18n-next:

```
// pages/farmer/dashboard.js
import { useTranslation } from 'next-i18next';

export default function Dashboard() {
  const { t } = useTranslation();
  return <h1>{t('dashboard.title')}</h1>;
}

// public/locales/en/common.json
{
  "dashboard": {
    "title": "Dashboard",
```

```
    "welcome": "Welcome, {{name}}"
  }
}
```

12. Backend Implementation

12.1 Express Server Structure

```
// server.js
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');

dotenv.config();
const app = express();

// Middleware
app.use(cors());
app.use(express.json());

// Routes
app.use('/api/auth', require('./routes/auth'));
app.use('/api/prices', require('./routes/prices'));
app.use('/api/recommendation',
require('./routes/recommendation'));
app.use('/api/buyers', require('./routes/buyers'));

// Error handling
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Internal server error' });
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port
${PORT}`));
```

12.2 Database Connection Pool

```
// db.js
const { Pool } = require('pg');

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  max: 20,
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 2000,
});

module.exports = pool;
```

12.3 API Route Example

```
// routes/prices.js
const express = require('express');
const router = express.Router();
const db = require('../db');
const redis = require('redis');

const client = redis.createClient();
```

```
// GET /api/prices/:crop
router.get('/:crop', async (req, res) => {
  const { crop } = req.params;
  const { days = 30 } = req.query;

  // Check cache first
  const cached = await client.get(`prices:${crop}`);
  if (cached) return res.json(JSON.parse(cached));

  // Query database
  const result = await db.query(
    'SELECT * FROM prices WHERE crop = $1 AND date >= NOW() -
INTERVAL \'1 day\' * $2 ORDER BY date DESC',
    [crop, days]
  );

  // Cache for 1 hour
  await client.setex(`prices:${crop}`, 3600,
JSON.stringify(result.rows));

  res.json(result.rows);
});

module.exports = router;
```

13. Machine Learning Model

13.1 Price Prediction Model

Approach: Linear Regression + Time Series Analysis

```
# train_price_model.py
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
import pickle

# Load historical price data
data = pd.read_csv('price_history.csv')

# Feature engineering
data['price_lag1'] = data['price'].shift(1)
data['price_lag7'] = data['price'].shift(7)
data['moving_avg7'] = data['price'].rolling(7).mean()
data['volatility'] = data['price'].rolling(7).std()
data['day_of_year'] = pd.to_datetime(data['date']).dt.dayofyear

# Split train/test
X = data[['price_lag1', 'price_lag7', 'moving_avg7', 'volatility',
'day_of_year']].dropna()
y = data['price'][len(X):]

X_train, X_test = X[:int(0.8*len(X))], X[int(0.8*len(X)):]
y_train, y_test = y[:len(X_train)], y[len(X_train):]

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Evaluate
train_score = model.score(X_train, y_train)
test_score = model.score(X_test, y_test)
print(f"Train R²: {train_score:.4f}, Test R²: {test_score:.4f}")

# Save model
pickle.dump(model, open('price_model.pkl', 'wb'))
```

13.2 Prediction Pipeline

```
# predict.py
import pickle
import pandas as pd
from datetime import datetime, timedelta

def predict_7day_prices(crop, current_price):
    """Generate 7-day price forecast"""

    # Load model
    model = pickle.load(open(f'models/{crop}_model.pkl', 'rb'))
```

```
    # Get recent history
    history = get_recent_prices(crop, days=30)

    # Feature engineering
    features = {
        'price_lag1': history[-1],
        'price_lag7': history[-7],
        'moving_avg7': np.mean(history[-7:]),
        'volatility': np.std(history[-7:]),
        'day_of_year': datetime.now().timetuple().tm_yday
    }

    # Generate predictions for next 7 days
    predictions = []
    for day in range(7):
        X = pd.DataFrame([features])
        pred = model.predict(X)[0]
        predictions.append(pred)

        # Update features for next day
        features['price_lag1'] = pred
        features['price_lag7'] = history[-(6-day)] if day < 6 else
pred
        features['day_of_year'] += 1

    return predictions
```

13.3 Model Evaluation Metrics

Metric	Target	Description
MAE	<200 Rs	Mean Absolute Error
RMSE	<250 Rs	Root Mean Squared Error
MAPE	<12%	Mean Absolute Percentage Error
R^2 Score	>0.75	Coefficient of determination

14. Deployment Guide

14.1 Docker Setup

Dockerfile for backend:

```
FROM node:16-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --production

COPY . .

EXPOSE 5000
CMD ["node", "server.js"]
```

Docker Compose (all services):

```
version: '3.8'
services:
  postgres:
    image: postgres:13
    environment:
      POSTGRES_DB: krishibandhu
      POSTGRES_PASSWORD: password
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  redis:
    image: redis:6-alpine
    ports:
      - "6379:6379"

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    depends_on:
      - postgres
      - redis
    environment:
      DATABASE_URL:
postgresql://postgres:password@postgres:5432/krishibandhu
      REDIS_URL: redis://redis:6379

  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
      - backend
```

```
volumes:
  postgres_data:
```

14.2 Heroku Deployment

Step 1: Create Heroku app

```
heroku login
heroku create krishibandhu-backend
```

Step 2: Add PostgreSQL addon

```
heroku addons:create heroku-postgresql:hobby-dev
```

Step 3: Deploy

```
git push heroku main
```

14.3 Vercel Deployment (Frontend)

Connect GitHub repository to Vercel, set environment variables, and auto-deploy on push.

```
vercel env add NEXT_PUBLIC_API_URL https://api.krishibandhu.com
vercel deploy
```

15. Testing Guide

15.1 Unit Tests (Backend)

```
// tests/auth.test.js
const request = require('supertest');
const app = require('../server');
const db = require('../db');

describe('Authentication', () => {
  test('POST /api/auth/register - should register new user', async
() => {
    const res = await request(app)
      .post('/api/auth/register')
      .send({
        email: 'farmer@test.com',
        password: 'password123',
        role: 'farmer'
      });

    expect(res.statusCode).toBe(201);
    expect(res.body).toHaveProperty('token');
  });

  test('POST /api/auth/login - should login user', async () => {
    const res = await request(app)
      .post('/api/auth/login')
      .send({
        email: 'farmer@test.com',
        password: 'password123'
      });

    expect(res.statusCode).toBe(200);
  });
});
```

15.2 Integration Tests

End-to-end testing of full workflows:

- User registration → login → view prices → get recommendation
- Buyer login → search farmers → place order → pay
- Farmer feedback on recommendation → model retraining

15.3 Performance Testing

Load testing with k6:

```
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  stages: [
    { duration: '30s', target: 20 },
    { duration: '1m30s', target: 100 },
```

```
    { duration: '30s', target: 0 },
  ],
};

export default function() {
  const res =
http.get('https://api.krishibandhu.com/prices/wheat');
  check(res, {
    'status is 200': (r) => r.status === 200,
    'response time < 500ms': (r) => r.timings.duration < 500,
  });
}
```

16. Troubleshooting

16.1 Common Issues & Solutions

Issue: Database connection timeout

- Causes: PostgreSQL not running, wrong connection string, firewall blocking
- Solution: Check PostgreSQL status, verify DATABASE_URL in .env, allow port 5432

Issue: Redis cache not working

- Causes: Redis server offline, wrong REDIS_URL
- Solution: Start Redis service, verify connection string

Issue: Mandi API returns 401 Unauthorized

- Causes: Invalid API key, API quota exceeded
- Solution: Verify API key, check rate limits, request quota increase

Issue: ML model prediction very inaccurate

- Causes: Insufficient training data, concept drift, model outdated
- Solution: Retrain model with recent data, implement monitoring, weekly retraining

Issue: Frontend fails to load after deployment

- Causes: Environment variables not set, API endpoint wrong
- Solution: Set NEXT_PUBLIC_API_URL in Vercel, test API connectivity

16.2 Logging & Monitoring

Set up centralized logging:

```
// In Express server
const winston = require('winston');

const logger = winston.createLogger({
  level: 'info',
  format: winston.format.json(),
  transports: [
    new winston.transports.File({ filename: 'error.log', level:
'error' }),
    new winston.transports.File({ filename: 'combined.log' })
  ]
});

app.use((req, res, next) => {
  logger.info(`${req.method} ${req.url}`);
  next();
});
```

17. Future Enhancements

17.1 Phase 2 Features

Voice Support: Full voice interface for illiterate farmers using speech recognition

SMS Integration: Send price alerts & recommendations via SMS for low-data users

Offline Mode: Store essential data locally, sync when connected

Farmer Cooperatives: Enable group selling for better collective bargaining

Insurance Integration: Link with crop insurance providers based on risk scores

Supply Chain Tracking: Track produce from farm to buyer with IoT sensors

Loan Access: Connect to microfinance based on trust score & price history

Sustainability Tracking: Carbon footprint, organic certification tracking

17.2 Expansion Plans

Geographic expansion:

- Phase 1 (Current): Nashik, Maharashtra
- Phase 2 (Q3 2024): Pune, Ahmednagar districts
- Phase 3 (Q4 2024): All of Maharashtra
- Phase 4 (2025): Gujarat, Madhya Pradesh, Rajasthan

Crop expansion:

- Phase 1: Wheat, Jowar, Sugarcane
- Phase 2: Cotton, Soybean, Maize
- Phase 3: Vegetables, Fruits (with cold storage)
- Phase 4: Spices, Pulses

18. Team Information

Name	Role	Expertise
Pushpak Mali	Full-Stack Development	React, Node.js, Database
Vedant Birpan	ML & Backend	Python, Data Science, APIs
Tanvi Mahajan	Frontend & UI/UX	React, UI Design, Mobile
Rutuja Deshmukh	DevOps & Deployment	Docker, CI/CD, Cloud

Contact: techtitans@rcpit.ac.in

GitHub: https://github.com/techtitans/krishibandhu

19. References

- Agmarknet Portal - https://agmarknet.gov.in (Government of India mandi data)
- FAO Agricultural Price Data - https://www.fao.org
- OpenWeather API - https://openweathermap.org/api
- Kaggle Agricultural Datasets - https://kaggle.com
- Research: "Deep Learning for Agricultural Price Prediction" - IEEE
- React.js Documentation - https://reactjs.org
- Node.js Best Practices - https://nodejs.org/en/docs/
- PostgreSQL Documentation - https://www.postgresql.org/docs/
- scikit-learn ML Guide - https://scikit-learn.org
- Docker & Kubernetes - https://www.docker.com